



Rapport
MABS Projet 2

Présenté par :
CHAUSSIEZ Benoit
LEMONNIER Paul

Master 1 **Méthodes informatiques et
statistiques pour les omics (MISO)**

Année universitaire : 2025-2026

Sommaire

1) Traitement des données	3
1.1) Filtrage des données par score de qualité	3
1.2) Implémentation du filtrage	4
1.3) Extraction des k-mers	5
1.4) Implémentation et matériels	6
2) Table des Suffixes	6
2.1) Implémentation de la table	6
2.2) Utilisation de la table	7
3) Alignement	7
3.1) Extraction de la région candidate	7
3.2) Alignement semi-global	7
3.3) Calcul de l'identité	8
3.4) Sélection du meilleur alignement	8
3.5) Détermination du brin	8
4) Résultat	8
4.1) Résultat sur les gros fichiers	8
4.2) Résultat sur les 10 000 premiers reads	9
5) Perspectives et axes d'évolution	10

1) Traitement des données

1.1) Filtrage des données par score de qualité

Les données de séquençage sont fournies sous forme de paires de fichiers (paired-end) : SRR10971381_1.fastq.gz et SRR10971381_2.fastq.gz, correspondant respectivement aux lectures forward et reverse.

Après examen préliminaire des données de séquençage, nous avons observé la présence de nombreux reads constitués uniquement de nucléotides non identifiés ("NNN") associés à un score de qualité nul. Ces reads ne présentent aucun intérêt biologique et constituent un bruit de fond susceptible de fausser les résultats d'alignement.

Afin de ne conserver que les reads présentant un intérêt biologique, nous avons appliqué un filtrage basé sur la qualité des séquences. Le seuil retenu est un score de qualité moyen supérieur à 20 (Q20). Ce choix repose sur plusieurs considérations :

Le seuil Q20 est largement utilisé dans la communauté scientifique comme indicateur de qualité fiable. Il correspond à une probabilité d'erreur de séquençage de 1 % par base, soit une précision de 99 %.

Ce seuil correspond à un compromis parfait entre le fait d'avoir des reads de qualités sans être trop contraignant ne permettant de ne garder aucun reads . Ce dernier permet d'éliminer uniquement les mauvais reads qui peuvent altérer l'analyse . Malgré le fait que nous supprimons des reads, leur suppression permet quand même une analyse robuste de l'échantillon et une identification du pathogène.

Certains reads sont résiduels à des artefacts techniques et leur suppression est primordiale. Ces reads d'artefacts sont représentés par des reads composés d'un ensemble de NNN avec un score nul, ils peuvent venir d'erreur de séquençage , à des artefacts d'élaboration de librairie .

En éliminant la plupart des reads non significatifs, la probabilité de faux positif sur l'identification du virus décroît alors la probabilité de se fourvoyer sur le virus devient faible .

Cette garantie sur la qualité minimale des reads va améliorer la confiance sur les résultats sur le virus

Ce filtre sur les mauvais reads a permis de supprimer une bonne partie de READ passant d'un fichier compressé d'une taille de 1.1 et 1.2 Go à un fichier de taille plus petit avec 889 Mo et 839 Mo en fichier compressé .

Soit 113 131 856 ligne dans le fichier Fastq contenant les reads bruts à 57 351 760 lignes dans le fichier après filtrage donc plus de 55 780 096 ligne cela représente 13 945 024 reads supprimés et économisé en temps de calcul sur la table d'indexation et qui n'ont pas un score supérieur à 20 .

Le traitement des fichiers a été effectué par le fichier python Preprocess.py avec la fonction `Filter_fastq` ayant comme paramètre un fichier d'entrée par exemple `SRR10971381_1.fastq.gz` puis un fichier de sortie par exemple `SRR10971381_1_qualite20.fastq.gz` et un qualité minimal qui est égal à 20 ici .

1.2) Implémentation du filtrage

Afin de lire le fichier contenant les reads pour faire du traitement derrière, il faut utiliser la librairie biopython avec `SeqIO` qui permet de récupérer plusieurs choses et de parser le fichier cela va permettre de récupérer la séquence plus tard mais surtout de récupérer le score qualité .

Tout d'abord nous avons utilisé la fonction `Filter_Fastq` dans le fichier `preprocess.py` qui ouvre le fichier d'entrée et de sortie pour ensuite récupérer la qualité dans le parser donné par `SeqIO` .

Par la suite nous calculons la moyenne des scores de qualités pour ensuite faire la comparaison avec le score minimum souhaité si il est atteint on récupère le read dans le fichier de sortie sinon on passe souvent .

A chaque fois que 100 000 reads sont atteints, une ligne apparaît afin de montrer la progression sur le terminal afin d'afficher le nombre de read lu, la vitesse du nombre de read par seconde et le temps passé depuis le lancement du programme

Exemple de sortie :

Reads: 100,000 | Vitesse: 125,000/s | Temps: 1s

Reads: 200,000 | Vitesse: 130,000/s | Temps: 2s

Reads: 300,000 | Vitesse: 128,000/s | Temps: 2s

Terminé: 1,234,567 reads traités, 987,654 conservés en 9.8s

1.3) Extraction des k-mers

Après le traitement des données et supprimer les reads peu significatif, il faut récupérer les variables obligatoires afin de faire la table des suffixes.

Pour cela, il faut aller dans le fichier `DecoupeKmer.py` et regarder la seule fonction `extraire_kmers_read_biopython`, il a 4 paramètres, les fichiers d'entrée et de sortie, la taille de kmer souhaitée et la taille du pas souhaité.

Dans notre problématique, seule une faible fraction des reads correspond au pathogène et les reads d'intérêt sont peu nombreux par conséquent le temps de traitement doit être optimisé à grande échelle.

Par conséquent, pour la taille des k-mers nous avons choisi de prendre des k-mers de taille 25, cette taille permet un bon compromis entre la spécificité et la sensibilité. Mais après avoir essayé avec une taille 25 sur les jeux de données, cela a sorti aucun résultat étant donné que 25 engendre une trop grande spécificité sur les échantillons donc dans les faits on va plutôt prendre un k-mer de taille 15.

Par conséquent, pour l'ensemble des analyses présentées dans ce rapport, nous avons utilisé une taille de k-mer de 15 avec un pas de 5.

En plus avec un pas égal à 5 permet d'éviter des redondances et surtout de n'avoir des k-mers qui sont à 80 % identiques cela permet un gain de temps considérable étant donné que 4 k-mer ne sont pas à traiter tout en gardant une bonne sensibilité.

d'environ 89% selon nos tests sur données simulées, tout en divisant le temps de calcul par 5 de façon optimale.

Par exemple avec un pas égal à 5 :

Read: ATCGATCGATCGATCGATCG...

|---k=15---|

|---k=15---|

|---k=15---|

|---k=15---|

1.4) Implémentation et matériels

Ayant des ressources limitées de part l'utilisation d'une Virtual Machine, de 4 cœurs et 8 Go de RAM, l'utilisation d'un pas de 5 va permettre de réduire le nombre de seeds utilisés simultanément et permettre de réduire la mémoire allouée ou bien limiter son utilisation ce qui est important pour le passage à plus grande échelle avec les fichiers de 5 Go.

Ce paramétrage a été retenu car il représente le meilleur compromis entre performance et précision pour notre problématique d'identification virale sur un échantillon où le pathogène est minoritaire.

2) Table des Suffixes

2.1) Implémentation de la table

La table des suffixes est générée depuis un génome dans un fichier fasta. Après avoir lu le dit fichier, la table des suffixes est implémentée comme une liste composée d'entiers allant de 0 à la longueur du génome - 1 pour éviter les collisions. Cette liste est insérée dans une variable par ordre décroissant car étant une table des suffixes, la position du suffixe le plus court étant la dernière lettre du génome. Cette table est ensuite triée via une fonction lambda, ce qui permet de trier chaque suffixe en fonction de sa séquence et de sa longueur.

2.2) Utilisation de la table

Une fois créée, la table permet de rechercher si un kmer est présent dans un génome indiqué. Grâce à une recherche dichotomique, si le kmer est présent, les positions sont récupérées et peuvent être utilisées pour essayer d'aligner les reads sur le génome.

3) Alignement

L'alignement des reads sur le génome est réalisé selon une stratégie de type *seed-and-extend*. Après identification de positions candidates grâce à la table des suffixes, une étape d'extension permet de valider ou non l'alignement.

3.1) Extraction de la région candidate

Pour chaque position candidate issue du seeding, une sous-région du génome est extraite autour de cette position. Cette fenêtre est définie de manière à inclure le read ainsi qu'une marge supplémentaire afin de tolérer des insertions, délétions ou erreurs de séquençage.

3.2) Alignement semi-global

L'alignement est effectué à l'aide de la bibliothèque Parasail, qui implémente des algorithmes d'alignement optimisés (SIMD).

Dans cette implémentation, la fonction `sg_trace` est utilisée, ce qui correspond à un alignement semi-global permettant :

- de ne pas pénaliser les extrémités du read
- de gérer efficacement les insertions et délétions

Chaque read est testé dans les deux orientations :

- brin direct (+)
- complément inverse (-)

3.3) Calcul de l'identité

Après alignement, une mesure d'identité est calculée en comparant les séquences alignées :

- seules les positions sans gap sont considérées
- le pourcentage d'identité est calculé comme le ratio : nombre de matches / nombre total de positions comparées

Un alignement est retenu si cette identité dépasse un seuil fixé (par exemple 90 %).

3.4) Sélection du meilleur alignement

Pour chaque read, plusieurs positions candidates peuvent être testées. L'alignement retenu est celui ayant :

- le meilleur score d'alignement
- et satisfaisant le seuil d'identité

La position finale est estimée à partir du début de l'alignement dans la région extraite.

3.5) Détermination du brin

Le brin d'origine du read est déterminé en fonction de l'orientation (directe ou complément inverse) ayant produit le meilleur score d'alignement.

4) Résultat

4.1) Résultat sur les gros fichiers

Au début, on a essayé de faire de faire l'indexation et l'alignement sur l'ensemble des reads, on a essayé sur le fichier SRR10971381_1_qualite20.fastq.gz avec le génome du coronavirus.

Les paramètres utilisés étaient un k-mer de taille 15 avec un pas de 5 et un pourcentage d'identité de 80%.

Exemple du résultat :

Résultats écrits dans result_CORO_1.tsv

Alignement: 92084/14337940 (0.64%)

- Brin direct (+): 27765

- Brin inverse (-): 64319

Cependant, cette analyse a nécessité 7 heures de calcul. Il n'est donc pas envisageable, dans le temps imparti, de réaliser ce travail sur l'ensemble des génomes avec les deux fichiers de reads.

Temps total estimé :

Si on fait un petit calcul, 7h de calcul pour un fichier, il y a 4 génome sur 2 fichier, on doit faire 56 heures d'alignement non stop donc cela n'est pas possible.

En revanche, nous avons observé qu'en se limitant aux 10 000 premiers reads par fichier, nous obtenons suffisamment de données pour distinguer les différences entre les échantillons et les génomes. Cette approche permet de réduire drastiquement le temps de calcul tout en conservant une capacité de comparaison pertinente.

4.2) Résultat sur les 10 000 premiers reads

Étant donné que les reads sont aléatoirement répartis dans les fichiers, le fait de prendre uniquement 10 000 reads n'est pas handicapant sur la conclusion. Il est plutôt représentatif de la diversité de l'échantillon .

Tableau récapitulatif des résultats:

Génome	Résultat sur SRR10971381_1	Temps d'exécution	Résultat sur SRR10971381_2	Temps d'exécution	Total des alignements	Moyenne des alignements
HIV	Strand + 3 Strand - 15	8.24 secondes	Strand + 23 Strand - 2	8.29 secondes	43	0.215%
Rhinovirus	Strand + 1 Strand - 1	8.87 secondes	Strand + 1 Strand - 0	7.59 secondes	3	0.015%
Coronavirus	Strand+20 Strand- 48	15.7 secondes	Strand + 75 Strand - 18	15.33 secondes	161	0.805%

Grippe	Strand + 4 Strand - 2	8.74 secondes	Strand + 3 Strand - 1	8.55 secondes	10	0.05%
--------	--------------------------	------------------	--------------------------	------------------	----	-------

On peut conclure que sur les 2 fichiers de READ les résultats les plus concluants sont pour le coronavirus , qui est le plus logique étant donnée la problématique du devoir .

5) Perspectives et axes d'évolution

Ce travail pratique met en lumière plusieurs limitations dans notre arsenal de programmation qu'il serait intéressant d'explorer pour aller plus loin dans ce travail.

L'un des principaux freins identifiés est le temps de calcul prohibitif (7 heures par alignement complet, soit 56 heures pour l'ensemble des combinaisons avec 4 génomes sur 2 fichiers de read). Nous pensons que l'utilisation de Python bien que pratique pour le prototypage et aussi l'un des seul langages qui nous sont. Il constitue un facteur aggravant, car ce langage peut introduire des lourdeurs d'exécution, notamment lors de la manipulation de grandes quantités de données.

Sans changer de langage de programmation, une piste d'amélioration majeure serait l'implémentation d'un multiprocessing. Notre algorithme actuel traite les reads séquentiellement sur un seul cœur. En découpant le fichier de reads et en répartissant les blocs sur plusieurs cœurs du processeur, nous pourrions réduire drastiquement le temps d'exécution en coupant les fichiers de reads en 2 cœurs cela passerait de 7h à 3h30 4h, en tenant compte de l'overhead de gestion des processus et de l'agrégation des résultats.

Cette approche permettrait de conserver la logique actuelle tout en exploitant pleinement les ressources matérielles disponibles avec la VM à 4 cœurs.

Face à ces contraintes, nous avons fait le choix pragmatique de nous limiter aux 10 000 premiers reads par fichier. Si cette décision nous a permis de respecter les délais et d'obtenir des résultats exploitables, elle constitue néanmoins un compromis

méthodologique. Une parallélisation efficace aurait probablement rendu possible l'analyse complète des données sans sacrifier la profondeur de l'échantillonnage.

Ce travail souligne l'importance d'anticiper les enjeux de performance dès la conception d'un pipeline d'analyse bioinformatique. Dans une version future, l'intégration du multiprocessing voire le passage à un langage plus performant (C, Rust) serait à envisager pour allier robustesse algorithmique et efficacité computationnelle.

L'intégration d'une étape de preprocessing dans notre pipeline s'est avérée bénéfique pour réduire la durée des alignements. Avec environ 28 000 000 de read sur le fichier brut et 14 000 000 de read dans le fichier après preprocess cela représente une économie de moitié de read non contributif donc la moitié du temps d'économiser soit 7h pour le fichier avec preprocess donc environ 14h pour le fichier brut sans preprocess. Sachant que le preprocessing demande environ 1h pour les 2 fichier donc on a une économie d'environ 12h à 13h pour les deux fichiers.

La meilleure option afin de diminuer le temps et de faire en sorte de gagner un maximum de temps serait de faire un multiprocessing sur le preprocessing afin de supprimer les reads avec un quality score inférieur à 20 sur les deux fichiers en même temps pour gagner une heure. Ensuite faire du multiprocessing sur l'algorithme et les fichiers de génomes pour passer de 7h pour un fichier à 7 heure pour 2 fichier, soit 29 heure pour tout les fichiers avec preprocessing à la place de 114 heure avec les fichiers bruts et les l'alignements sur un seul coeur (14 heure pour un fichier avec 4 génome et 2 fichier à faire sur chaque génome).